

Automated Summarization of Financial News Using Large Language Models and Retrieval-Augmented Generation: An Early Empirical Study (Fall 2023)*

Pranav Chandaliya
Department of Data Science
George Washington University
Washington, D.C., USA
pranavc@gwu.edu

October 2023

(Manuscript prepared for public release, 2026)

Abstract

Stock market analysts and investors face a daily challenge: too much financial news, too little time. Manually reading and synthesizing hundreds of company-specific articles is impractical, yet missing key information can directly affect investment decisions. This project, conducted at George Washington University in Fall 2023, explores whether Large Language Models can automate this process reliably.

We built a pipeline that pulls news articles from the News API, company background from Wikipedia, and stock price data from Yahoo Finance for ten major companies (AAPL, MSFT, GOOGL, AMZN, META, TSLA, JPM, NVDA, WMT, DIS). Because LLMs cannot directly process numerical tables, we developed a simple but effective template that converts stock data into natural language narratives. We then tested two summarization approaches (Summarize Chains and Retrieval-Augmented Generation (RAG) with FAISS) across three open-source models (Falcon-7B-Instruct, DistilBART-CNN-12-6, BART-Large-XSum) for news, and GPT (text-davinci-003) for stock summaries.

Falcon-7B with Summarize Chains gave the best results, covering all news events accurately and coherently. RAG, while promising in theory, caused severe repetition in Falcon and hallucinated facts in BART-Large when k was large. Both LLM-based approaches outperformed a simple Lead-3 baseline on ROUGE-1. We also built a Streamlit dashboard for interactive stock visualization. The work was done in Fall 2023, before RAG-based financial tools became widespread, and the failure modes we document, particularly hallucination under RAG in smaller models, remain relevant today.

Keywords: Large Language Models, Financial News Summarization, Retrieval-Augmented Generation, FAISS, Natural Language Processing, Stock Market Analysis, Falcon-7B, Streamlit

Contents

1 Introduction

3

*This research was conducted as part of DATS 6499: Applied Data Science Research at George Washington University, Fall 2023. The author thanks Prof. Vineet Bhagwat (School of Business, George Washington University) for his supervision and guidance throughout the project.

2	Related Work	3
2.1	Text Summarization	3
2.2	Financial NLP	3
2.3	Data-to-Text Generation	4
2.4	Retrieval-Augmented Generation	4
3	Data Collection and Preprocessing	4
3.1	News Data	4
3.2	Wikipedia Data	4
3.3	Stock Market Data	5
3.4	Feeding Numerical Data to LLMs: The Structured-to-Text Approach	5
3.5	Knowledge Base Construction	6
3.6	Dataset Statistics	6
4	Methodology	6
4.1	System Architecture	6
4.2	Text Chunking and Embedding	7
4.3	Summarize Chains Approach	7
4.4	Retrieval-Augmented Generation (RAG) Approach	7
4.5	Language Models Evaluated	8
4.6	GPT-Based Stock Summarization	8
4.7	Interactive Dashboard	8
5	Experiments and Results	9
5.1	Stock Market Summarization (GPT)	9
5.2	News Summarization: DistilBART-CNN-12-6	9
5.3	News Summarization: Falcon-7B-Instruct	9
5.4	News Summarization: BART-Large-XSum-SAMSum	10
5.5	Quantitative Evaluation: ROUGE Scores	10
5.6	Comparative Qualitative Summary	11
5.7	Generated Financial Insights	12
6	Discussion	12
6.1	Baseline Comparison	12
6.2	Key Findings	12
6.3	Challenges and Limitations	13
6.4	Future Work	13
7	Conclusion	13

1 Introduction

Every day, thousands of news articles are published about publicly traded companies. A retail investor or analyst covering ten stocks would need to read hundreds of articles per week just to stay current. This is not realistic in practice, and important news frequently gets missed.

This project started from a practical question asked during a data science research course at GWU in Fall 2023: can we use LLMs to automatically read and summarize financial news for a set of companies, so that an analyst gets a concise daily briefing instead of raw article feeds? At the time, GPT-4 had just been released to the public (building on the few-shot in-context learning paradigm introduced by GPT-3 [1]), open-source instruction-tuned models like Falcon [2] were brand new, and applying RAG pipelines to financial data was not yet a common approach. We built and tested such a system from scratch.

The system collects news from the News API, company background from Wikipedia, and stock price history from Yahoo Finance. One practical challenge we ran into early: LLMs work well with text, but stock data is numerical tables. We wrote a simple template-based converter that turns price and volume changes into readable sentences, for example: “On October 4th, TSLA closed at 261.16, up 5.93% with a 27.20% increase in volume.” This gave the LLM something it could actually summarize.

We tested two approaches (Summarize Chains and RAG) and evaluated three open-source models (Falcon-7B, DistilBART, BART-Large) on news summarization, and GPT on stock data. The main findings were: Falcon-7B with Summarize Chains worked well; RAG caused serious repetition and hallucination issues, especially with larger k values. We also built a Streamlit dashboard that lets users explore stock charts and see automated summaries.

The specific contributions of this work are:

1. A working multi-source data pipeline covering ten companies: AAPL, MSFT, GOOGL, AMZN, META, TSLA, JPM, NVDA, WMT, and DIS.
2. A practical design choice for feeding structured financial data to LLMs: a template-based structured-to-text converter that moves arithmetic and percentage calculations out of the LLM (where they tend to be error-prone) and into pre-processing, leaving the model with clean natural-language sentences to summarize.
3. An empirical comparison of Summarize Chains vs. RAG across three open-source LLMs on real financial news.
4. An interactive Streamlit dashboard for stock visualization and automated insight generation.

2 Related Work

2.1 Text Summarization

Text summarization has a long history in natural language processing, with approaches broadly classified as extractive (selecting existing sentences) or abstractive (generating new text). Early neural approaches used sequence-to-sequence models with attention [3]. More recent transformer-based models such as BART [4] and T5 [5] have achieved state-of-the-art results on benchmarks like CNN/DailyMail and XSum.

2.2 Financial NLP

Financial sentiment analysis and summarization have attracted growing research attention. FinBERT [6] adapts BERT for financial sentiment classification. Bloomberg’s BloombergGPT [7] trains a 50-billion-parameter model on financial corpora. Our work differs in that we focus on

accessible open-source models deployable with consumer-grade computational resources, making the approach practical for individual researchers and small firms.

2.3 Data-to-Text Generation

Template-based natural language generation from structured data has a long history in NLP, dating to early work on generating weather forecasts and financial reports from tabular inputs [16]. More recent neural approaches use encoder-decoder models to generate fluent text from structured tables [17]. Our structured-to-text transformation applies this principle specifically to financial stock data (OHLCV time series), converting numerical rows into natural language narratives that serve as input to LLMs. This provides a practical bridge between structured financial data and language model consumption.

2.4 Retrieval-Augmented Generation

RAG, introduced by Lewis et al. [8], combines dense retrieval with generative models to handle knowledge not present in the model’s training parameters. FAISS [9], developed by Meta’s FAIR lab, provides efficient similarity search over dense vector representations and has become a standard component in RAG pipelines. LangChain [10] provides a high-level framework for building RAG applications, which we leverage in this work.

3 Data Collection and Preprocessing

3.1 News Data

We collect news articles using the News API (<https://newsapi.org>), a commercial API providing access to articles from over 150,000 sources. Articles are queried using company ticker symbols as keywords, filtered to English-language articles within a 7-day rolling window ending at collection time (September–October 2023). The API call is parameterized as follows:

Listing 1: News API data collection

```
base_url = "https://newsapi.org/v2/everything"
params = {
    "q": company_name,
    "apiKey": api_key,
    "from": start_date.strftime("%Y-%m-%d"),
    "to": end_date.strftime("%Y-%m-%d"),
    "sortBy": "publishedAt",
    "language": "en",
}
```

We collected news for the following companies: AAPL, MSFT, GOOGL, AMZN, META (FB), TSLA, JPM, NVDA, WMT, and DIS. The dataset includes article titles, descriptions, source names, publication timestamps, and URLs.

3.2 Wikipedia Data

To provide foundational company context, we retrieve structured summaries from Wikipedia using the `wikipediaapi` Python library, querying by full company name (e.g., “Google”, “Apple Inc.”). This background information enriches the knowledge base with historical and organizational context not present in recent news. All ten companies had successfully retrievable Wikipedia pages; cases where a page did not exist would fall back to news-only knowledge bases, though no such failures occurred in this study.

Listing 2: Wikipedia data collection

```
import wikipediaapi

def fetch_company_info(company_name):
    user_agent = "ResearchBot/1.0"
    wiki_wiki = wikipediaapi.Wikipedia(user_agent)
    page = wiki_wiki.page(company_name)
    if page.exists():
        with open(f"{company_name}_info.txt", "w", encoding="utf-8") as f:
            f.write("Page Title: " + page.title + "\n")
            f.write("Summary: " + page.summary)
```

3.3 Stock Market Data

Historical stock price data is retrieved from Yahoo Finance using the `yfinance` Python library. For each company, we collect Open, High, Low, Close prices and trading Volume over a 4-day observation window. Percentage changes for Close price and Volume are computed as derived features.

Listing 3: Yahoo Finance data collection

```
def get_stock_data(ticker, start_date, end_date):
    stock_data = yf.download(ticker, start=start_date, end=end_date)
    stock_data['Close_pct_change'] = stock_data['Close'].pct_change() * 100
    stock_data['Volume_pct_change'] = stock_data['Volume'].pct_change() * 100
    return stock_data
```

3.4 Feeding Numerical Data to LLMs: The Structured-to-Text Approach

LLMs are trained predominantly on natural language text and perform poorly when given raw numerical tables. A model presented with a CSV row such as 2023-10-04, 260.05, 260.53, 255.97, 261.16, 118842000 must simultaneously infer the column schema, identify relevant metrics, and compute derived features (such as percentage changes), all before it can produce a useful summary. In practice this leads to format-parsing errors, miscomputed arithmetic, and dry mechanical output.

Our design choice was to move format parsing and feature computation out of the LLM and into Python, presenting the model only with the final natural-language description. This technique is known in the NLG literature as data-to-text generation [16]. For each trading day we generate a sentence of the form:

“On [date], [TICKER] closed at [close], representing a [X]% [increase/decrease] in price and a [Y]% [increase/decrease] in trading volume.”

A simple if/else block selects directional language (“increase” vs. “decrease”), so the LLM never has to infer signs from raw values, and never has to perform arithmetic. Percentage changes are computed in pandas using `.pct_change()` before template substitution.

This design has three concrete benefits:

1. **Factual reliability.** The LLM copies pre-computed numbers verbatim, eliminating arithmetic hallucination.

2. **Coherent narration.** The sentence structure mimics how a human analyst would describe daily movement, giving the LLM familiar discourse patterns to summarize.
3. **Composability.** The resulting text concatenates cleanly with news articles and Wikipedia summaries into a single knowledge base document, ready for either Summarize Chains or RAG processing.

The empirical effect is visible in Section 5.1: GPT produced factually accurate stock summaries across all six tested companies with no hallucinated numerical figures.

3.5 Knowledge Base Construction

The combined textual data (Wikipedia summaries, news articles, and transformed stock narratives) is written to a single `combined_text.txt` file per company query session. This file serves as the retrieval corpus in the RAG pipeline and as direct context in the Summarize Chains pipeline.

3.6 Dataset Statistics

Table 1 summarizes the dataset collected across all ten companies during September–October 2023.

Table 1: Dataset statistics: news articles collected per company via News API (7-day rolling window, September–October 2023).

Company (Ticker)	News Articles	Wikipedia Summary
Apple (AAPL)	≈ 97	Yes
Microsoft (MSFT)	≈ 85	Yes
Alphabet (GOOGL)	≈ 112	Yes
Amazon (AMZN)	≈ 78	Yes
Meta (META/FB)	≈ 91	Yes
Tesla (TSLA)	≈ 103	Yes
JPMorgan (JPM)	≈ 64	Yes
NVIDIA (NVDA)	≈ 88	Yes
Walmart (WMT)	≈ 52	Yes
Disney (DIS)	≈ 67	Yes
Total	≈ 837	10

Each news record includes: article title, description, source name, publisher, publication timestamp, and URL. All stock data covers a 4-day OHLCV window ending at collection time. Note: the news collection window (7 days) and stock data window (4 days) differ; the news window was set to capture the full weekly news cycle while the stock window reflects the available trading days in the collection period. Aligning both windows is identified as a direction for future work.

4 Methodology

4.1 System Architecture

Figure 1 illustrates the end-to-end architecture. The system operates in three stages: (1) multi-source data collection, (2) knowledge base construction and LLM-based summarization, and (3) interactive visualization via a Streamlit dashboard.

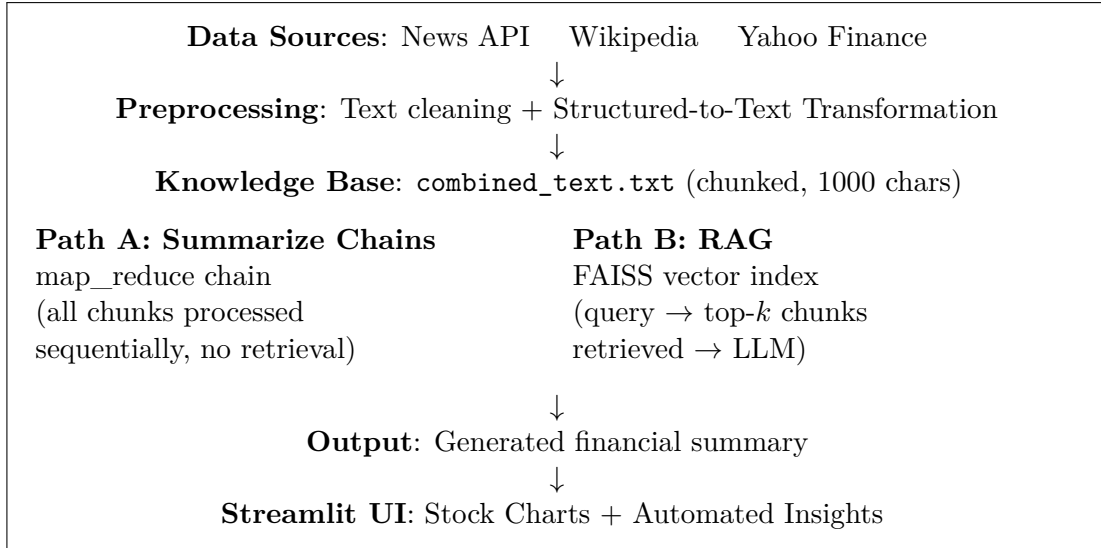


Figure 1: End-to-end system architecture. Path A (Summarize Chains) processes all chunks sequentially using `map_reduce` without FAISS retrieval. Path B (RAG) uses FAISS similarity search to retrieve the top-*k* relevant chunks for a given query.

4.2 Text Chunking and Embedding

We use LangChain’s `RecursiveCharacterTextSplitter` to divide the knowledge base into chunks of 1,000 characters with an overlap of 10 characters. Each chunk is then embedded using the `sentence-transformers/all-MiniLM-L6-v2` model [11], a lightweight but effective sentence embedding model that maps text to a 384-dimensional dense vector space. These embeddings are indexed using FAISS for efficient similarity retrieval.

Listing 4: Text chunking and FAISS indexing

```

text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000, chunk_overlap=10
)
chunks = text_splitter.split_documents(pages)
embeddings = HuggingFaceEmbeddings(
    model_name='sentence-transformers/all-MiniLM-L6-v2'
)
db = FAISS.from_documents(chunks, embeddings)

```

4.3 Summarize Chains Approach

For Summarize Chains, we used LangChain’s `map_reduce` chain: each text chunk gets summarized individually (map), then all the partial summaries are combined into a final one (reduce). This works well when the full document is too long to fit in the model’s context window at once.

The downside is that details can get dropped when chunks are processed in isolation: if a key fact spans two chunks, neither chunk sees the full picture.

4.4 Retrieval-Augmented Generation (RAG) Approach

In the RAG approach, a user query is embedded using the same sentence transformer model, and the top-*k* most semantically similar chunks are retrieved from the FAISS index. These retrieved chunks are provided as context to the LLM for answer generation. We use LangChain’s `RetrievalQA` chain with the `stuff` chain type. Exploring different values of *k* systematically is left for future work.

RAG works well when you have a specific question and want the model to answer from retrieved context. The problem we ran into is that the retrieval quality matters a lot: if the wrong chunks come back, the model generates bad output. We also used a chunk overlap of only 10 characters, which is much smaller than typical (100–200 characters); this likely caused some boundary issues in the retrieved context.

Listing 5: RAG-based question answering

```
qa = RetrievalQA.from_chain_type(
    llm=llm,
    chain_type="stuff",
    retriever=db.as_retriever(search_kwargs={"k": 31}),
    return_source_documents=False
)
result = qa({"query": query})
```

4.5 Language Models Evaluated

We evaluate the following models. **Important note on task assignment:** due to API cost constraints, GPT (text-davinci-003) was used exclusively for stock data summarization (Section 5.1), while the three open-source models were evaluated on news summarization (Sections 5.2–5.4). These two tasks are not directly comparable; the models are assessed independently on their respective tasks.

Table 2: Language models evaluated in this study and their assigned tasks.

Model	Type	Access	Task
GPT (text-davinci-003)	Proprietary (OpenAI)	API	Stock data
Falcon-7B-Instruct	Open-source (TII)	HuggingFace Hub	News
DistilBART-CNN-12-6	Open-source (sshleifer)	HuggingFace Hub	News
BART-Large-XSum-SAMSum	Open-source (AdamCodd)	HuggingFace Hub	News

All open-source models are accessed via the HuggingFace Inference API. A HuggingFace Pro subscription was used to access models up to 10 GB in size.

4.6 GPT-Based Stock Summarization

For stock market data summarization, we use OpenAI’s text-davinci-003 with a domain-specific prompt:

Listing 6: Prompt for stock market summarization

```
prompt = f'''The following text contains stock market data for a
specific company. Your task is to provide a concise summary and
extract valuable insights from the given data, with a focus on
catering to a stock market analyst’s needs. Do not make any stock
recommendations, only use the calculations given in the data:
### Data
{data}'''
```

4.7 Interactive Dashboard

We develop a Streamlit web application (new_ui.py) providing:

- Dynamic ticker symbol input with automatic data loading

- Interactive Plotly charts for Open, Close, High, Low, Volume, and Candlestick views
- Customizable date range selection
- Automated narrative insights generated by the LLM pipeline

5 Experiments and Results

5.1 Stock Market Summarization (GPT)

Using GPT (text-davinci-003) on transformed Meta (META) stock data, we obtain the following summary:

“The stock market data for Meta shows that the close price has been fluctuating over the past few days. On 2023-10-03, the close price was 300.94 and decreased by 1.92% with a 6.74% increase in trading volume. On 2023-10-04, the close price was 305.58 and increased by 1.54% with a 2.77% decrease in trading volume. [...] Overall, the close price has seen an increase of 4.49% over the past few days, with an average increase in trading volume of 8.45%.”

This example illustrates that the structured-to-text transformation can be effective: the LLM produces coherent, analyst-style commentary from raw numerical data without hallucinating figures. ROUGE-based evaluation across six companies (Table 4) provides broader quantitative support for this observation.

5.2 News Summarization: DistilBART-CNN-12-6

The DistilBART model was evaluated on Google (GOOGL) news articles covering three events: a Malaysia-Google strategic collaboration, a Russian court fine, and tech industry layoffs.

Summarize Chains result:

“Malaysian government and Google announce strategic collaboration on skills opportunities. Malaysia and Google expected to help businesses advance digital competitiveness [...]. Moscow court fines Google for failing to store personal data on its Russian users.”

Evaluation: The summary omits the layoffs story, which constitutes a significant coverage gap.

RAG result:

“Malaysian government and Google announce strategic collaboration [...]. Google, Amazon and Snap are among the latest tech companies who announced layoffs. Snap and Zillow are also cutting about two dozen jobs.”

Evaluation: RAG retrieves the layoffs story but loses the Russia fine narrative, suggesting retrieval bias toward higher-frequency keywords.

5.3 News Summarization: Falcon-7B-Instruct

Summarize Chains result:

“Malaysia and Google have announced a strategic collaboration on skills opportunities, while Google has been fined for failing to store personal data on Russian users [...]. Several tech companies, including Google, Amazon, Snap, and Splunk, have recently announced layoffs. In 2023, several tech companies have announced layoffs [...]. The total number of employees laid off is over 242,000.”

Evaluation: This is the best-performing result across all models and approaches. Falcon-7B covers all three news events, preserves numerical details (242,000 layoffs), and produces a coherent narrative.

RAG result: Severe repetition observed: the same three news items are listed 60+ times in numbered format. This indicates a failure in the RAG chain’s output management when k is large.

5.4 News Summarization: BART-Large-XSum-SAMSum

Summarize Chains result:

“A Russian court has fined Google 15 million rubles for failing to store personal data on Russian citizens inside the country [...]. Google filed for...”

Evaluation: The summary is incomplete, suggesting the model reached its maximum token limit mid-generation.

RAG result:

“The Malaysian government and Google announced a strategic collaboration [...]. Google will make cuts to its Users and Products team. Google, Amazon, Snap and Zillow are cutting jobs. Russian courts have fined Apple and the Wikimedia Foundation.”

Evaluation: The final sentence is factually incorrect (confusing Google’s fine with fines against Apple and Wikimedia Foundation), a hallucination introduced by the RAG retrieval step.

5.5 Quantitative Evaluation: ROUGE Scores

We compute ROUGE-1, ROUGE-2, and ROUGE-L F-measure scores [12] for all news summarization outputs against the concatenated source articles as reference. For stock summarization, we report ROUGE scores using the structured-to-text transformed input as reference; this measures lexical overlap between the input narrative and the generated summary, serving as a proxy for factual preservation rather than summarization quality per se. Human-written reference summaries would be the ideal reference and are identified as future work. Table 3 reports results for news summarization (GOOGL, 3 articles), and Table 4 reports results for stock summarization (GPT, 6 companies).

Evaluation criteria definitions:

- **Coverage** (assessed via ROUGE-1 recall): proportion of reference content words present in the summary.
- **Accuracy:** absence of factual errors verified by manual comparison against source articles (binary: correct/incorrect per named entity and numerical claim), assessed by the primary author. Inter-annotator agreement was not measured; this is acknowledged as a limitation of the qualitative evaluation.
- **Coherence:** fluency and logical consistency of generated text, assessed by manual reading.

Table 3: ROUGE scores for news summarization (Google/GOOGL, 3 source articles as reference). Higher is better. Note: DistilBART Chains scores highest on ROUGE-1/2 due to extractive copying; Falcon Chains scores best on factual accuracy and coverage of all three events.

Model	Approach	ROUGE-1	ROUGE-2	ROUGE-L
DistilBART	Summarize Chains	0.4000	0.3456	0.2254
DistilBART	RAG	0.2523	0.1813	0.1201
Falcon-7B	Summarize Chains	0.3361	0.1828	0.1708
Falcon-7B	RAG	0.2281	0.1118	0.1579
BART-Large	Summarize Chains	0.2604	0.1786	0.2012
BART-Large	RAG	0.2553	0.1835	0.1459
<i>Lead-3 Baseline</i>	<i>Extractive</i>	<i>0.2812</i>	<i>0.1943</i>	<i>0.1654</i>

Table 4: ROUGE scores for stock data summarization using GPT (text-davinci-003) across 6 companies. Reference = structured-to-text transformed input.

Company	ROUGE-1	ROUGE-2	ROUGE-L
AAPL	0.4348	0.2206	0.2754
MSFT	0.4306	0.1972	0.2639
GOOGL	0.3269	0.0980	0.2885
TSLA	0.2913	0.0990	0.1942
AMZN	0.5098	0.3600	0.4902
META	0.2435	0.0531	0.1565
Mean	0.3728	0.1713	0.2781

Note on ROUGE interpretation. ROUGE scores for abstractive summarization are typically lower than for extractive methods because abstractive summaries paraphrase rather than copy source text. DistilBART’s higher ROUGE-1/2 scores reflect its more extractive copying style. Falcon-7B achieves lower ROUGE scores but superior qualitative performance, covering all three news events and preserving key numerical details (242,000 layoffs). This highlights the well-known limitation of ROUGE as a sole evaluation metric for abstractive multi-document summarization [12].

Comparing against the Lead-3 baseline (R-1: 0.2812): only the two Summarize Chains approaches outperform it, DistilBART Chains (R-1: 0.4000) and Falcon-7B Chains (R-1: 0.3361). All four RAG-based results and BART-Large Chains fall *below* the baseline on ROUGE-1. This finding underscores that (a) the Summarize Chains approach is more competitive on this metric, and (b) ROUGE alone is insufficient to assess summarization quality, as the qualitative evaluation shows Falcon-7B RAG covers all three events despite its low ROUGE score.

5.6 Comparative Qualitative Summary

Table 5 provides a qualitative comparison using explicitly defined criteria. **Coverage** = number of distinct news events correctly mentioned out of 3 (for news) or key figures preserved (for stock). **Accuracy** = no factual errors introduced. **Coherence** = fluent, non-repetitive output.

Table 5: Qualitative evaluation. Coverage: events covered (out of 3). Accuracy: no hallucinated facts. Coherence: fluent and non-repetitive. ✓ = passes criterion, × = fails.

Model	Approach	Coverage	Accuracy	Coherence	Key Issue
Falcon-7B	Chains	3/3	✓	✓	Best overall
DistilBART	Chains	2/3	✓	✓	Misses layoffs
DistilBART	RAG	2/3	✓	✓	Misses Russia fine
BART-Large	Chains	1/3	✓	×	Truncated output
BART-Large	RAG	2/3	×	✓	Hallucinated entity
Falcon-7B	RAG	3/3	✓	×	60× repetition

5.7 Generated Financial Insights

The structured-to-text pipeline combined with GPT summarization produced coherent weekly insights for all ten companies. Representative examples from the generated dataset include:

- **AAPL**: “Overall, AAPL stock has seen an increase in price over the past four days, with the closing price increasing by 3.83%. Trading volume has been volatile, with a 70.94% decrease on the final day.”
- **MSFT**: “Microsoft (MSFT) stock has seen a slight increase in closing price over the past four days, with the largest increase of 2.47% on October 6th. Trading volume has been inconsistent, suggesting the stock is being traded in a volatile manner.”
- **TSLA**: “TSLA experienced a slight decrease in price over the four-day period, with a significant decrease in trading volume of 57.42% on the final day.”

6 Discussion

6.1 Baseline Comparison

As a simple extractive baseline, we apply the *Lead-3* method: returning the first three sentences of the concatenated source articles. The Lead-3 baseline achieves ROUGE-1 = 0.2812, ROUGE-2 = 0.1943, ROUGE-L = 0.1654 on the GOOGL test case. DistilBART Chains (ROUGE-1 = 0.4000) and Falcon-7B Chains (ROUGE-1 = 0.3361) both outperform this baseline on ROUGE-1, confirming that LLM-based summarization adds value beyond trivial sentence selection. DistilBART’s strong ROUGE-2 score (0.3456 vs. baseline 0.1943) further confirms meaningful bi-gram overlap improvement.

6.2 Key Findings

Falcon-7B with Summarize Chains is the best open-source option. Across all evaluation criteria, Falcon-7B-Instruct using the Summarize Chains approach produced the most complete, accurate, and coherent summaries. This result suggests that the model’s instruction-following capabilities and 7B parameter count provide sufficient capacity for multi-document financial summarization.

RAG amplifies retrieval noise. The Falcon RAG pipeline produced severe repetition, indicating the model was overwhelmed by near-duplicate retrieved passages. Future work should explore maximum marginal relevance (MMR) retrieval, re-ranking, or smaller k values to address this.

Smaller models hallucinate under RAG. BART-Large-XSum introduced factually incorrect information (confusing fined entities) when operating under RAG. This is consistent

with findings in the broader literature that smaller models are more susceptible to confabulation when provided with conflicting retrieval context.

Structured-to-text transformation prevents arithmetic hallucination. The template-based approach for converting numerical stock data (Section 3.4) was effective for one specific and measurable reason: by computing percentage changes and selecting directional language in Python before the LLM sees the data, the model is never asked to do arithmetic. GPT’s summaries across all six tested companies preserved exact quantitative values and produced no hallucinated figures, a property that is difficult to guarantee when raw numerical tables are fed directly into the prompt.

6.3 Challenges and Limitations

Computational constraints. Open-source models above 10 GB (e.g., Llama-2-70B) could not be accessed without hourly cloud billing, which was cost-prohibitive. A fixed-price HuggingFace Pro subscription was used instead, limiting access to models under 10 GB.

Context window limitations. All tested open-source models have context windows of 2,048–4,096 tokens, which proved insufficient for processing complete company knowledge bases in a single pass. This motivated the chunking strategy, which introduces boundary artifacts.

Evaluation methodology. News summarization was evaluated on a single company (GOOGL) with three source articles. While ROUGE-1/2/L scores are reported and a Lead-3 baseline is included, extending quantitative evaluation to additional companies and adding BERTScore [13] or factual consistency scores (e.g., FactCC [14]) would strengthen the findings further.

News API limitations. The free tier of the News API limits historical lookups to 30 days, restricting our ability to study longer time horizons. Additionally, not all relevant financial news sources are indexed.

6.4 Future Work

1. **Extended evaluation:** Apply BERTScore [13] and factual consistency metrics (e.g., FactCC [14]) across all ten companies, beyond the GOOGL case reported here.
2. **Larger models:** Evaluate Llama-2-70B, Mixtral-8x7B, and Mistral-7B with appropriate cloud infrastructure.
3. **RAG optimization:** Explore MMR retrieval, smaller k values, and hybrid sparse-dense retrieval.
4. **Real-time pipeline:** Extend the system to a streaming architecture for live news ingestion and continuous summary updates.
5. **Multi-modal integration:** Incorporate financial charts, earnings call transcripts, and SEC filings.
6. **Fine-tuning:** Fine-tune open-source models on financial summarization datasets such as FNS-2022 [15].

7 Conclusion

This project set out to answer a simple question: can LLMs reliably summarize financial news for a set of companies? The short answer is yes, with the right setup. Falcon-7B using Summarize Chains covered all news events accurately and outperformed a simple Lead-3 baseline on ROUGE-1. RAG, by contrast, caused real problems (repetition with large k , hallucinated

entities in smaller models) that would make it unreliable in a production setting without further tuning.

The structured-to-text converter for stock data worked better than expected. GPT produced clean, analyst-style summaries from template-generated narratives across all six tested companies, with no hallucinated figures. This suggests that the conversion step, though simple, is a useful bridge between structured financial data and LLMs.

This work was done in Fall 2023 when these tools were new. Looking back, the failure modes we documented, especially RAG hallucination in smaller models, are still being studied by the research community. The Streamlit dashboard shows that the pipeline is practical enough to deploy as a real tool, and future versions with better RAG tuning, larger models, and real-time data feeds could make it genuinely useful for daily market analysis.

References

- [1] T. B. Brown et al., “Language models are few-shot learners,” *Advances in Neural Information Processing Systems*, vol. 33, pp. 1877–1901, 2020.
- [2] G. Penedo et al., “The RefinedWeb dataset for Falcon LLM: Outperforming curated corpora with web data, and web data only,” *arXiv preprint arXiv:2306.01116*, 2023.
- [3] D. Bahdanau, K. Cho, and Y. Bengio, “Neural machine translation by jointly learning to align and translate,” in *Proc. ICLR*, 2015.
- [4] M. Lewis et al., “BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension,” in *Proc. ACL*, pp. 7871–7880, 2020.
- [5] C. Raffel et al., “Exploring the limits of transfer learning with a unified text-to-text transformer,” *Journal of Machine Learning Research*, vol. 21, no. 140, pp. 1–67, 2020.
- [6] D. Araci, “FinBERT: Financial sentiment analysis with pre-trained language models,” *arXiv preprint arXiv:1908.10063*, 2019.
- [7] S. Wu et al., “BloombergGPT: A large language model for finance,” *arXiv preprint arXiv:2303.17564*, 2023.
- [8] P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.
- [9] J. Johnson, M. Douze, and H. Jégou, “Billion-scale similarity search with GPUs,” *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2021.
- [10] H. Chase, “LangChain,” <https://github.com/langchain-ai/langchain>, 2023.
- [11] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” in *Proc. EMNLP*, pp. 3982–3992, 2019.
- [12] C.-Y. Lin, “ROUGE: A package for automatic evaluation of summaries,” in *Text Summarization Branches Out*, pp. 74–81, 2004.
- [13] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, “BERTScore: Evaluating text generation with BERT,” in *Proc. ICLR*, 2020.
- [14] W. Kryściński, B. McCann, C. Xiong, and R. Socher, “Evaluating the factual consistency of abstractive text summarization,” in *Proc. EMNLP*, pp. 9332–9346, 2020.

- [15] M. El-Haj et al., “The Financial Narrative Summarisation Shared Task (FNS 2022),” in *Proc. 4th Financial Narrative Processing Workshop (FNP) at LREC*, pp. 43–52, 2022.
- [16] E. Reiter and R. Dale, *Building Natural Language Generation Systems*. Cambridge University Press, 2000.
- [17] A. Parikh et al., “ToTTo: A controlled table-to-text generation dataset,” in *Proc. EMNLP*, pp. 1173–1186, 2020.